Tecnicatura Superior en Desarrollo de Software

Módulo 07 — InterfaceAdapter: `ViewModel` y View

Actividad Práctica — CU-CLI-001: Registrar Cliente | Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

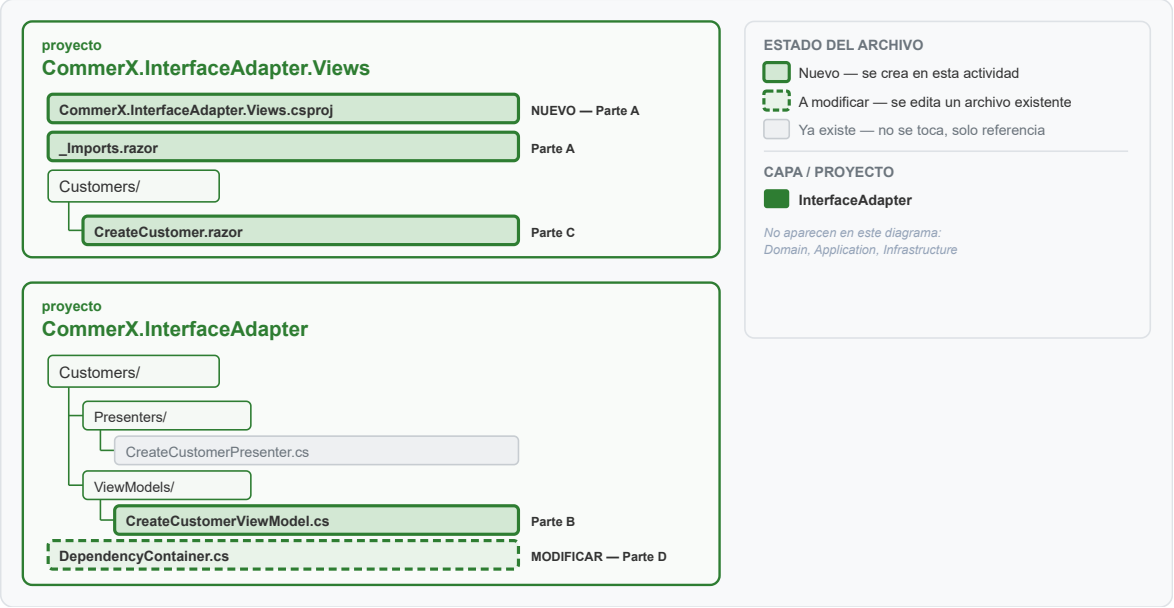
Contexto de la actividad

En la primera parte del Módulo 07 construiste `CreateCustomerPresenter`: la clase que recibe las notificaciones del caso de uso a través de `ICreateCustomerOutputPort` y las acumula en un `OperationResult<CreateCustomerResponse>`. Ese Presenter todavía no tiene quién lo consuma: nadie arma el `CreateCustomerRequest` y nadie muestra el resultado en pantalla.

En esta actividad completás ese circuito con dos piezas: un **ViewModel** (`CreateCustomerViewModel`), que orquesta la llamada al caso de uso y traduce el resultado del Presenter a estado listo para mostrar, y una **View** (`CreateCustomer.razor`), un componente Blazor que hace *binding* contra ese ViewModel. Para que una View compile y funcione hace falta antes un proyecto propio, de un tipo especial que hasta ahora no habías usado (Parte A).

Esta actividad **no construye el host de escritorio** (WPF con Blazor Hybrid), que es quien va a mostrar la pantalla. Como todavía no hay host, vas a comprobar que la View funciona con una prueba automática que la renderiza y la maneja como lo haría un usuario (Parte E).

Archivos involucrados



Todos los archivos pertenecen a la capa InterfaceAdapter (verde), repartidos en dos proyectos: el que ya tenías y el nuevo proyecto de Views. Los de borde sólido son **NUEVOS**, los de borde discontinuo se **MODIFICAN** y el gris ya existe. La letra de cada archivo es la de la Parte donde lo trabajás.

Parte A — Crear el proyecto de Views (biblioteca de clases de Razor)

Ubicación

`src/InterfaceAdapter.Views/` (proyecto nuevo, dentro de la carpeta de solución `3.InterfaceAdapter`)

Descripción

Una View es un archivo `.razor` (HTML con código C# mezclado). Hasta ahora todos tus proyectos solo tenían archivos `.cs`, y un proyecto común de C# **no sabe compilar archivos** `.razor`: los ignora en silencio. La compilación termina con 0 errores y 0 advertencias, pero la View nunca existió. Por eso las Views van en un proyecto propio, de un tipo especial: una **biblioteca de clases de Razor** (*Razor Class Library*).

Podría pensarse que alcanza con agregar los archivos `.razor` a un proyecto que ya existe. Técnicamente se puede (cambiando su SDK y agregando el paquete de Blazor), pero separarlas en su propio proyecto, una **biblioteca de clases de Razor** (*Razor Class*

Library), tiene cuatro razones concretas:

1. **Es lo que sabe compilar un `.razor`.** Un proyecto común de C# ignora los `.razor` sin avisar: termina con 0 errores y la View no existe. La plantilla de biblioteca de clases de Razor viene con el SDK correcto, el paquete de Blazor y el archivo de importaciones ya configurados, así que no queda ningún paso manual que se pueda olvidar.
2. **Mantiene limpia la arquitectura.** El proyecto `CommerX.InterfaceAdapter` (Presenters y ViewModels) queda en C# puro, sin depender de Blazor. Se puede probar, y reutilizar, sin ninguna tecnología de pantalla. Si mañana la interfaz cambiara de tecnología, solo se reemplaza el proyecto de Views.
3. **Es una pieza pensada para ser referenciada.** Una biblioteca de clases de Razor es el formato estándar para compartir componentes: el host (la aplicación de escritorio con Blazor Hybrid) simplemente la referencia y muestra sus pantallas. Además puede llevar consigo recursos estáticos (hojas de estilo, scripts, imágenes en `wwwroot`) que viajan con la biblioteca.
4. **Deja los límites visibles en la solución.** La dirección de la dependencia queda escrita en una referencia de proyecto: Views referencia a InterfaceAdapter, y nunca al revés.

La contrapartida es honesta: hay **un proyecto más** en la solución. Se paga con un archivo de proyecto adicional y una referencia, y a cambio cada proyecto tiene una sola responsabilidad.

> **Dirección de la dependencia:** el proyecto de Views referencia a InterfaceAdapter (para usar los ViewModels), nunca al revés. InterfaceAdapter no sabe que existen las Views.

Paso 1 — Crear el proyecto

Opción 1 — Línea de comandos (desde la carpeta raíz de la solución, donde está `CommerX.slnx`):

```
dotnet new razorclasslib -n CommerX.InterfaceAdapter.Views -o src/InterfaceAdapter.Views -f net9.0
dotnet sln CommerX.slnx add src/InterfaceAdapter.Views/CommerX.InterfaceAdapter.Views.csproj --solution-folder 3.InterfaceAdapter
```

1. **Comando 1:** `dotnet new` crea un proyecto a partir de una plantilla; `razorclasslib` es la plantilla "biblioteca de clases de Razor". `-n` es el nombre, `-o` la carpeta donde se crea y `-f net9.0` el framework (el mismo que el resto de la solución).
2. **Comando 2:** agrega el proyecto a la solución, dentro de la carpeta de solución `3.InterfaceAdapter`.

Opción 2 — Visual Studio:

1. En el Explorador de soluciones, clic derecho sobre la carpeta `3.InterfaceAdapter` → **Agregar** → **Nuevo proyecto…**
2. Buscá la plantilla **Biblioteca de clases de Razor** (*Razor Class Library*) y elegila.
3. Nombre: `CommerX.InterfaceAdapter.Views`. Ubicación: la carpeta `src` de la solución. Si el asistente ofrece *Admitir páginas y vistas* (*Support pages and views*), dejalo sin marcar. Framework: **.NET 9.0**.

Paso 2 — Qué generó la plantilla

Abrí el archivo `CommerX.InterfaceAdapter.Views.csproj`. Esto es lo que trae la plantilla (más, al final, la referencia que agregás en el paso 4):

```
1  <Project Sdk="Microsoft.NET.Sdk.Razor">
2
3    <PropertyGroup>
4      <TargetFramework>net9.0</TargetFramework>
5      <Nullable>enable</Nullable>
6      <ImplicitUsings>enable</ImplicitUsings>
7    </PropertyGroup>
8
9
10   <ItemGroup>
11     <SupportedPlatform Include="browser" />
12   </ItemGroup>
13
14   <ItemGroup>
15     <PackageReference Include="Microsoft.AspNetCore.Components.Web" Version="9.0.20" />
16   </ItemGroup>
17
18   <ItemGroup>
19     <ProjectReference Include="..\InterfaceAdapter\CommerX.InterfaceAdapter.csproj" />
20   </ItemGroup>
21
22 </Project>
```

Cómo leerlo:
Línea 1: `Microsoft.NET.Sdk.Razor` es el "kit" de compilación que entiende los archivos `.razor`. Es la diferencia con los demás proyectos (que usan `Microsoft.NET.Sdk`).

Línea 11: declara que la biblioteca es compatible con Blazor (la plataforma *browser*); la agrega la plantilla y no hay que tocarla.

Línea 15: el paquete NuGet con los tipos de Blazor (`OwningComponentBase`, los eventos `@onclick` y `@bind`…). Es el mismo tipo de referencia que agregaste en el Módulo 08

para Entity Framework Core: versión exacta, rama 9.x porque la solución apunta a `net9.0`.

Además la plantilla generó archivos de ejemplo que **no vamos a usar**: `Component1.razor`, `Component1.razor.css`, `ExampleJsInterop.cs` y la carpeta `wwwroot`. Eliminalos (clic derecho → **Eliminar** en el Explorador de soluciones): dejar código de ejemplo es dejar código muerto en el proyecto.

Paso 3 — Referenciar el proyecto InterfaceAdapter

La View necesita usar `CreateCustomerViewModel`, que vive en `CommerX.InterfaceAdapter`. Eso se declara con una **referencia a proyecto** (el otro tipo de referencia, distinta de un paquete NuGet: acá el código es de tu misma solución):

```
dotnet add src/InterfaceAdapter.Views/CommerX.InterfaceAdapter.Views.csproj reference src/InterfaceAdapter/CommerX.InterfaceAdapter.csproj
```

En Visual Studio: clic derecho sobre **Dependencias** del proyecto de Views → **Agregar referencia de proyecto…** → marcá `CommerX.InterfaceAdapter`. En el `.csproj` aparece la línea 19 (`ProjectReference`).

Paso 4 — Completar el archivo _Imports.razor

En un archivo `.cs` cada archivo empieza con sus `using`. Un componente Razor también puede llevar sus `@using` arriba, pero en la práctica se repetirían en todas las Views. Para eso existe `_Imports.razor`: un archivo de texto con los `@using` que comparten los componentes. Estas son las reglas para usarlo correctamente:

- **El nombre es exacto:** `_Imports.razor` (guion bajo, *I* mayúscula, plural). Con cualquier otro nombre, el compilador lo trata como un componente más y no lo aplica.
- **Se aplica solo, sin referenciarlo:** no hay que invocarlo desde ninguna View. El compilador de Razor lo detecta y agrega esos `@using` a todos los `.razor` de *su carpeta y de las subcarpetas*.
- **Puede haber más de uno:** uno en la raíz del proyecto para lo común, y otros dentro de una carpeta para lo que solo necesitan sus componentes. Los de la raíz y los de la carpeta se acumulan.
- **Qué va ahí:** la línea de eventos web que trae la plantilla (`@using Microsoft.AspNetCore.Components.Web`) y los namespaces de los tipos que usan tus Views, como los de los ViewModels (`@using CommerX.InterfaceAdapter.Customers.ViewModels`). **Qué no va:** lógica ni código.
- **La línea de eventos web nunca se borra:** sin ella la View compila, pero `@bind` y `@onclick` no responden.

- **Si cambia un namespace, se actualiza acá:** si movés un ViewModel a otro namespace, hay que corregir la línea correspondiente del `_Imports.razor` (en las Views no hay nada que tocar).

Cómo diagnosticar cuando una View dice que no encuentra un tipo, en este orden: (1) ¿el `@using` del namespace de ese tipo está en el `_Imports.razor`? (2) ¿el proyecto de Views referencia al proyecto que contiene ese tipo? (3) ¿está instalado el paquete NuGet que lo define? Casi siempre es la primera.

Así se ve la diferencia en una View. Sin `_Imports.razor`, cada componente tendría que repetir las líneas de importación; con él, empieza directamente por lo que lo hace único:

Sin `_Imports.razor` — cada View repetiría las importaciones:

```
@using Microsoft.AspNetCore.Components.Web
@using CommerX.InterfaceAdapter.Customers.ViewModels
@inherits OwningComponentBase<CreateCustomerViewModel>

<h3>CU-CLI-001 — Alta de Cliente</h3>
<!-- ...los campos... -->
```

Con `_Imports.razor` — la View empieza directamente por lo suyo:

```
@inherits OwningComponentBase<CreateCustomerViewModel>

<h3>CU-CLI-001 — Alta de Cliente</h3>
<!-- ...los campos... -->
```

En tu proyecto, la plantilla ya crea el archivo con la primera línea y vos agregás la segunda. Así queda:

```
@using Microsoft.AspNetCore.Components.Web
@using CommerX.InterfaceAdapter.Customers.ViewModels
```

Qué está pasando:
Línea 1 (la trae la plantilla): los *eventos* de HTML (`@onclick`, `@bind`). Es la que más importa: **sin ella la View compila sin errores**, pero `@bind` y `@onclick` no hacen nada: el botón se ve y no responde. Nunca la borres.

Línea 2 (la agregás vos): el namespace de tus ViewModels, para poder escribir `CreateCustomerViewModel` en la View sin prefijo.

> **Comprobá:** ejecutá `dotnet build` en la carpeta de la solución. Todavía no hay View, así que tiene que terminar con 0 errores y 0 advertencias. Que la View de verdad funcione se comprueba en la Parte E.

Parte B — Crear `CreateCustomerViewModel.cs`

Ubicación

`src/InterfaceAdapter/Customers/ViewModels/CreateCustomerViewModel.cs`

Descripción

Este ViewModel recibe el caso de uso (a través de `ICreateCustomerInputPort`) y el Presenter que ya construiste, ambos por constructor. Expone una propiedad pública por cada campo del formulario, todas de tipos simples y listas para `@bind`, más `Mensaje` y `EnCurso` para reflejar el resultado de la operación.

```
using CommerX.Application.Customers.DTOs;
using CommerX.Application.Customers.Ports;
using CommerX.InterfaceAdapter.Customers.Presenters;

namespace CommerX.InterfaceAdapter.Customers.ViewModels;

// clase sellada — orquesta la llamada al UseCase, no valida reglas de negocio
public sealed class CreateCustomerViewModel
{
    private readonly ICreateCustomerInputPort _useCase;
    private readonly CreateCustomerPresenter _presenter;

    // propiedades bindables — una por campo del formulario CU-CLI-001
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string Document { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Phone { get; set; } = string.Empty;
    public string Address { get; set; } = string.Empty;
    public DateOnly BirthDate { get; set; } = DateOnly.FromDateTime(DateTime.Today);

    // estado de resultado — lo lee la View después de GuardarAsync()
    public string Mensaje { get; private set; } = string.Empty;
    public bool EnCurso { get; private set; }

    // el contenedor de DI inyecta ambas dependencias desde el mismo scope de pantalla
    public CreateCustomerViewModel(
        ICreateCustomerInputPort useCase,
        CreateCustomerPresenter presenter)
    {
        _useCase = useCase;
        _presenter = presenter;
    }

    // comando que la View dispara desde el botón "Guardar"
    public async Task GuardarAsync()
```

```
37      {
38          EnCurso = true;
39
40          try
41          {
42              // construye el Request a partir del estado bindable — DTO inmutable, object in
    itializer
43              var request = new CreateCustomerRequest
44              {
45                  FirstName = FirstName,
46                  LastName = LastName,
47                  Document = Document,
48                  Email = Email,
49                  Phone = Phone,
50                  Address = Address,
51                  BirthDate = BirthDate
52              };
53
54              await _useCase.ExecuteAsync(request);
55
56              // lee el Presenter — nunca el UseCase ni el OutputPort directamente
57              Mensaje = _presenter.Result!.IsSuccess
58                  ? $"Cliente '{FirstName} {LastName}' registrado correctamente."
59                  : string.Join("; ", _presenter.Result.Errors);
60          }
61          finally
62          {
63              // se libera siempre, aunque el caso de uso falle: así el botón nunca queda des
    habilitado
64              EnCurso = false;
65          }
66      }
67  }
```

Qué está pasando:
Línea 27: el constructor recibe el *puerto de entrada* (la interfaz del caso de uso) y la clase *concreta* `CreateCustomerPresenter`: el ViewModel necesita leer su propiedad `Resul t`, que la interfaz del puerto de salida no expone.

Línea 38: `EnCurso` se enciende al empezar, para que la View deshabilite el botón mientras se guarda.

Líneas 40 y 61: el `try` / `finally` garantiza que `EnCurso` vuelva a `false` *pase lo que pase*. Si el caso de uso lanzara una excepción inesperada (por ejemplo una falla de la base de datos), sin el `finally` el botón quedaría deshabilitado para siempre.

Línea 57: el mensaje se arma leyendo el Presenter, nunca el caso de uso. Si fue exitoso es un texto de confirmación; si no, se unen los errores que el Presenter acumuló.

> **El ViewModel no valida reglas de negocio:** no revisa si el email tiene formato correcto ni si el cliente es mayor de edad. Eso ya lo hacen Application y Domain, y el ViewModel solo muestra lo que ellos informaron.

Parte C — Crear `CreateCustomer.razor`

Ubicación

`src/InterfaceAdapter.Views/Customers/CreateCustomer.razor` (proyecto de Views)

Descripción

La View hereda de `OwningComponentBase<CreateCustomerViewModel>`: eso le abre un *scope* de inyección de dependencias propio a esta pantalla y le entrega el ViewModel en la propiedad `Service`, tal como viste en el Anexo 07e. Todo el binding apunta a `Service`; no hay ningún `@inject` suelto. No hace falta escribir `@using` acá arriba: ya están en `_Imports.razor` (Parte A, paso 4).

```razor
@* CreateCustomer.razor *@
@inherits OwningComponentBase<CreateCustomerViewModel>

<h3>CU-CLI-001 — Alta de Cliente</h3>

<label>Nombre</label>
<input @bind="Service.FirstName" />

<label>Apellido</label>
<input @bind="Service.LastName" />

<label>Documento</label>
<input @bind="Service.Document" />

<label>Email</label>
<input @bind="Service.Email" />

<label>Teléfono</label>
<input @bind="Service.Phone" />

<label>Domicilio</label>
<input @bind="Service.Address" />

<label>Fecha de nacimiento</label>
<input type="date" @bind="Service.BirthDate" />

```

```
27  <button @onclick="Service.GuardarAsync" disabled="@Service.EnCurso">
28      Guardar
29  </button>
30
31  <p>@Service.Mensaje</p>
```

Qué está pasando:
Línea 2: la herencia que abre el scope propio y expone el ViewModel como `Service`.

Línea 7: `@bind` conecta el campo con la propiedad del ViewModel en las dos direcciones. Por defecto actualiza la propiedad cuando el campo pierde el foco.

Línea 25: con `type="date"`, `@bind` convierte la fecha del navegador a `DateOnly` sin que escribas ninguna conversión.

Línea 27: el botón ejecuta `GuardarAsync` del ViewModel y se deshabilita mientras `EnCurso` sea verdadero.

Línea 31: muestra el mensaje que dejó el ViewModel.

> **Comprobá:** `dotnet build` tiene que terminar con 0 errores. Si el compilador dice que no encuentra `OwningComponentBase` o `CreateCustomerViewModel`, revisá el `_Imports.razor` y la referencia a InterfaceAdapter de la Parte A.

Parte D — Extender `DependencyContainer.cs`

Ubicación

`src/InterfaceAdapter/DependencyContainer.cs` (proyecto InterfaceAdapter, no el de Views)

Descripción

El `DependencyContainer` de InterfaceAdapter ya registra el Presenter (primera parte del Módulo 07). Sumás un método de extensión propio para el ViewModel, con el mismo ciclo de vida *Scoped*: así el ViewModel, el Presenter y el caso de uso quedan resueltos dentro del mismo scope de pantalla que abre `OwningComponentBase<T>`.

```
1  using CommerX.Application.Customers.Ports;
2  using CommerX.InterfaceAdapter.Customers.Presenters;
3  using CommerX.InterfaceAdapter.Customers.ViewModels;
4
5  // namespace truco — extiende IServiceCollection sin agregar using en Program.cs
```

```
6  namespace Microsoft.Extensions.DependencyInjection;
7
8  public static class DependencyContainer
9  {
10     public static IServiceCollection AddCustomerPresenter(
11         this IServiceCollection services)
12     {
13         services.AddScoped<CreateCustomerPresenter>();
14         services.AddScoped<ICreateCustomerOutputPort>(
15             provider => provider.GetRequiredService<CreateCustomerPresenter>());
16
17         return services;
18     }
19
20     // NUEVO — método propio para el ViewModel, mismo ciclo de vida que el Presenter
21     public static IServiceCollection AddCustomerViewModel(
22         this IServiceCollection services)
23     {
24         services.AddScoped<CreateCustomerViewModel>();
25
26         return services;
27     }
28 }
```

Qué está pasando:
Línea 21: método nuevo, al lado del que ya existía.

Línea 24: `AddScoped` hace que cada scope de pantalla reciba su propio ViewModel. Como el Presenter también es Scoped, el ViewModel y el caso de uso comparten *la misma instancia* del Presenter dentro de una pantalla, y dos pantallas abiertas nunca comparten estado.

Cuando exista el host, su arranque llamará a los tres métodos de registro: el de los casos de uso (`AddCustomerUseCases`), el del Presenter (`AddCustomerPresenter`) y este (`AddCustomerViewModel`).

Parte E — Comprobar la View sin host: una prueba que la renderiza

Ubicación

`tests/CommerX.InterfaceAdapter.Tests/Customers/Views/CreateCustomerViewTests.cs` (nuevo)

Descripción

Una View puede compilar y aun así no funcionar: los dos errores más comunes (poner la View en un proyecto común y borrar el `@using` de eventos) **no producen ningún error al compilar**. Como todavía no hay host que muestre la pantalla, la forma de comprobar que tu View responde es una prueba automática con **bUnit**, una librería que renderiza un componente Blazor en memoria y permite escribir en sus campos y hacer clic en sus botones.

Paso 1 — Agregar bUnit y la referencia al proyecto de Views

```
dotnet add tests/CommerX.InterfaceAdapter.Tests/CommerX.InterfaceAdapter.Tests.csproj package bunit --version 1.40.0
```

Es un paquete NuGet más, como los anteriores, pero va en el proyecto de **pruebas**, no en InterfaceAdapter ni en Views: la aplicación en producción no necesita bUnit. Además el proyecto de pruebas necesita ver la View, así que referencia al proyecto de Views (como en el paso 3 de la Parte A, pero desde las pruebas):

```
dotnet add tests/CommerX.InterfaceAdapter.Tests/CommerX.InterfaceAdapter.Tests.csproj reference src/InterfaceAdapter.Views/CommerX.InterfaceAdapter.Views.csproj
```

Paso 2 — Crear la prueba

Copiá este archivo tal cual. Arma la misma composición que va a armar el host (casos de uso, Presenter y ViewModel) con un repositorio en memoria en lugar de la base de datos:

```
using Bunit;
using CommerX.Domain.Customers.Entities;
using CommerX.Domain.Customers.Repositories;
using CommerX.InterfaceAdapter.Views.Customers;
using Microsoft.Extensions.DependencyInjection;
using Xunit;

namespace CommerX.InterfaceAdapter.Tests.Customers.Views;

// renderiza la View real (con el caso de uso real y un repositorio en memoria) y la maneja como lo haría un usuario
public sealed class CreateCustomerViewTests : TestContext
{
    private sealed class InMemoryCustomerRepository : ICustomerRepository
    {
        public readonly List<Customer> Stored = new();

        public Task<Customer?> FindByDocumentAsync(string document)
            => Task.FromResult(Stored.FirstOrDefault(c => c.Document.Value == document));

        public Task<Customer?> FindByEmailAsync(string email)
```

```
21              => Task.FromResult(Stored.FirstOrDefault(c => c.Email.Value == email));
22
23          public Task AddAsync(Customer customer)
24          {
25              Stored.Add(customer);
26              return Task.CompletedTask;
27          }
28
29          public Task<Customer?> GetByIdAsync(Guid id)
30              => Task.FromResult(Stored.FirstOrDefault(c => c.Id == id));
31
32          public Task UpdateAsync(Customer customer) => Task.CompletedTask;
33      }
34
35      private readonly InMemoryCustomerRepository _repository = new();
36
37      public CreateCustomerViewTests()
38      {
39          // la misma composición que armará el host: use case + presenter + view model
40          Services.AddSingleton<ICustomerRepository>(_repository);
41          Services.AddCustomerUseCases();
42          Services.AddCustomerPresenter();
43          Services.AddCustomerViewModel();
44      }
45
46      private static void FillForm(IRenderedComponent<CreateCustomer> view, string document =
    "12345678", string email = "juan@example.com")
47      {
48          // tras cada Change() el componente se vuelve a renderizar: hay que buscar el input
    de nuevo
49          view.FindAll("input")[0].Change("Juan");
50          view.FindAll("input")[1].Change("Pérez");
51          view.FindAll("input")[2].Change(document);
52          view.FindAll("input")[3].Change(email);
53          view.FindAll("input")[4].Change("+54 261 4123456");
54          view.FindAll("input")[5].Change("Av. San Martín 1000");
55          view.FindAll("input")[6].Change("1990-01-01");
56      }
57
58      [Fact]
59      public void Render_ShowsTheSevenFieldsAndTheSaveButton()
60      {
61          var view = RenderComponent<CreateCustomer>();
62
63          Assert.Equal(7, view.FindAll("input").Count);
64          Assert.Equal("Guardar", view.Find("button").TextContent.Trim());
65          Assert.Equal(string.Empty, view.Find("p").TextContent.Trim());
66      }
67
68      [Fact]
69      public void ClickingSave_WithValidData_RegistersTheCustomerAndShowsTheSuccessMessage()
70      {
```

```
71          var view = RenderComponent<CreateCustomer>();
72          FillForm(view);
73
74          view.Find("button").Click();
75
76          Assert.Equal("Cliente 'Juan Pérez' registrado correctamente.", view.Find("p").TextC
ontent.Trim());
77          var stored = Assert.Single(_repository.Stored);
78          Assert.Equal("12345678", stored.Document.Value);
79          Assert.False(view.Find("button").HasAttribute("disabled"));
80      }
81
82      [Fact]
83      public void ClickingSave_WithAnUnderageBirthDate_ShowsTheDomainErrorAndStoresNothing()
84      {
85          var view = RenderComponent<CreateCustomer>();
86          FillForm(view);
87          view.FindAll("input")[6].Change(DateTime.Today.AddYears(-10).ToString("yyyy-MM-d
d"));
88
89          view.Find("button").Click();
90
91          Assert.NotEqual(string.Empty, view.Find("p").TextContent.Trim());
92          Assert.DoesNotContain("registrado correctamente", view.Find("p").TextContent);
93          Assert.Empty(_repository.Stored);
94      }
95
96      [Fact]
97      public void ClickingSave_WithEmptyFields_ShowsValidationMessagesAndStoresNothing()
98      {
99          var view = RenderComponent<CreateCustomer>();
100
101         view.Find("button").Click();
102
103         Assert.NotEqual(string.Empty, view.Find("p").TextContent.Trim());
104         Assert.Empty(_repository.Stored);
105     }
106
107     [Fact]
108     public void ClickingSave_TwiceWithTheSameDocument_ShowsTheDuplicateMessageTheSecondTime
()
109     {
110         var view = RenderComponent<CreateCustomer>();
111         FillForm(view);
112         view.Find("button").Click();
113
114         view.FindAll("input")[3].Change("otro@example.com");
115         view.Find("button").Click();
116
117         Assert.Contains("12345678", view.Find("p").TextContent);
118         Assert.Single(_repository.Stored);
119     }
120
```

```
121     [Fact]
122     public void EachRenderedView_OwnsItsOwnViewModel()
123     {
124         var first = RenderComponent<CreateCustomer>();
125         var second = RenderComponent<CreateCustomer>();
126 
127         first.FindAll("input")[0].Change("Juan");
128 
129         Assert.Equal("Juan", first.FindAll("input")[0].GetAttribute("value"));
130         Assert.True(string.IsNullOrEmpty(second.FindAll("input")[0].GetAttribute("valu
    e")));
131     }
132 }
```

Paso 3 — Ejecutar las pruebas

```
1 dotnet test tests/CommerX.InterfaceAdapter.Tests
```

Qué verifica cada prueba:
Render_ShowsTheSevenFieldsAndTheSaveButton: la View se compiló (proyecto de Razor, Parte A) y muestra los siete campos y el botón.
ClickingSave_WithValidData...: los eventos funcionan (`_Imports.razor`, Parte A paso 4): escribir en los campos y hacer clic registra el cliente y muestra el mensaje.
ClickingSave_WithAnUnderageBirthDate... y ...WithEmptyFields...: los errores de dominio y de validación llegan a la pantalla y no se guarda nada.
ClickingSave_TwiceWithTheSameDocument...: el segundo intento muestra el aviso de documento duplicado.
EachRenderedView_OwnsItsOwnViewModel: cada pantalla abre su propio scope (`OwningComponentBase`): lo que se escribe en una no aparece en la otra.

> **Resultado esperado:** todas las pruebas en verde. Si alguna falla con *"The element does not have an event handler"*, falta la línea 1 de `_Imports.razor` (la que trae la plantilla: no la borres).

> **Cómo se reconoce el error en cada caso:** si la View estuviera en un proyecto común de C# (sin el SDK de Razor) no existiría y la prueba *ni siquiera compilaría* (no encuentra `CreateCustomer`); sin el `@using` de eventos, la prueba compila pero falla al escribir o hacer clic.

Checklist de verificación

Antes de continuar, verificá que cada ítem esté completo:

Archivo	Verificación
`CommerX.InterfaceAdapter.Views.csproj`	Proyecto creado con la plantilla de biblioteca de clases de Razor, dentro de la carpeta de solución `3.InterfaceAdapter`; usa `Microsoft.NET.Sdk.Razor`, referencia `Microsoft.AspNetCore.Components.Web` en 9.0.20 y el proyecto `CommerX.InterfaceAdapter`; sin los archivos de ejemplo de la plantilla; `dotnet build` sin errores ni advertencias
`_Imports.razor`	Conserva la línea de eventos web de la plantilla y agrega el `@using` del namespace de los ViewModels
`CreateCustomerViewModel.cs`	Recibe `ICreateCustomerInputPort` y `CreateCustomerPresenter` por constructor; expone los siete campos más `Mensaje` y `EnCurso`; `GuardarAsync()` arma el Request, ejecuta el caso de uso, lee `_presenter.Result` y libera `EnCurso` en un `finally`
`CreateCustomer.razor` (en el proyecto de Views)	Hereda `OwningComponentBase<CreateCustomerViewModel>`; todo el binding apunta a `Service`; sin ningún `@inject` suelto
`DependencyContainer.cs`	Nuevo método `AddCustomerViewModel` con `AddScoped<CreateCustomerViewModel>()`, junto a `AddCustomerPresenter`
`CreateCustomerViewTests.cs`	El proyecto de pruebas referencia al proyecto de Views y tiene bUnit 1.40.0; `dotnet test` deja todas las pruebas de la View en verde

Preguntas de reflexión

1. **¿Por qué `CreateCustomerViewModel` recibe la clase concreta `CreateCustomerPresenter` y no la interfaz `ICreateCustomerOutputPort`?**
   Pista: revisá qué expone cada uno y quién necesita leer `Result`.
2. **¿Qué pasaría si `CreateCustomer.razor` heredara de `OwningComponentBase<CreateCustomerViewModel>` pero además tuviera un componente hijo que heredara su propio `OwningComponentBase<CreateCustomerPresenter>`?**
   Pista: pensá cuántos scopes de inyección se abrirían y si seguirían compartiendo la misma instancia del Presenter. Repasá el Anexo 07e.

3. **Si `CreateCustomerViewModel` se pidiera con `@inject` directo en la View, en lugar de `OwningComponentBase<T>`, ¿qué pasaría la segunda vez que el usuario abre la pantalla de alta?**
   Pista: pensá qué instancia se resuelve y qué datos del intento anterior podrían seguir ahí.
4. **¿Por qué `GuardarAsync()` no valida el formato del email antes de llamar al caso de uso?**
   Pista: repasá dónde vive esa regla desde el Módulo 02 (Value Objects) y el Módulo 03 (excepciones de dominio).
5. **¿Qué diferencia hay entre el ViewModel de este módulo y el Presenter de la primera parte?**
   Pista: compará qué implementa cada uno, de quién recibe sus dependencias por constructor y a quién le pertenece conceptualmente el nombre "ViewModel".
6. **¿Por qué borrar la línea `@using Microsoft.AspNetCore.Components.Web` del `_Imports.razor` no produce ningún error al compilar, y qué te permite detectarlo?**
   Pista: sin ese `@using`, `@onclick` y `@bind` se tratan como atributos HTML comunes. ¿Qué parte de la actividad los ejecuta de verdad?
7. **¿Por qué las Views van en un proyecto aparte y no dentro de `CommerX.InterfaceAdapter`?**
   Pista: pensá qué hace un proyecto común con un archivo `.razor`, y qué proyectos quedan dependiendo de Blazor en cada alternativa.
8. **¿Qué le pasaría al botón "Guardar" si `GuardarAsync()` no tuviera el `finally` y la base de datos estuviera caída?**
   Pista: seguí el valor de `EnCurso` y de `disabled` paso a paso.